

PROCESSDATAAI

Document Ingestion and Semantic Search POC

A developer handout for understanding, running, rebuilding, and extending the application

At a glance A .NET 10 console application reads local PDFs, normalizes extracted text into Microsoft Data Ingestion documents, chunks that content, generates embeddings through Azure OpenAI or Ollama, stores vectors in memory, and returns the three most semantically similar chunks for a question.

Audience	Purpose
Developer new to the Microsoft AI ingestion stack	Understand the design and reproduce the implementation without prior library knowledge.
POC maintainer	Operate the application, interpret logs, diagnose ingestion failures, and identify production-hardening work.

Project root: C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI

Document version: July 2026

How to use this handout

Read Sections 1-4 for the conceptual model and architecture. Sections 5-13 explain the implementation file by file. Sections 14-17 cover operation, troubleshooting, limitations, and a practical rebuild checklist.

Contents

1. Application purpose and outcomes
2. Core concepts
3. Architecture and end-to-end flow
4. Project structure
5. Dependencies and why they exist
6. Startup and dependency injection
7. Environment configuration and validation
8. Embedding provider abstraction
9. PDF extraction and document normalization
10. Chunking strategy
11. Vector storage and chunk writing
12. Ingestion orchestration
13. Semantic search
14. Running and validating the application
15. Errors, logging, and troubleshooting
16. Limitations and production evolution
17. Reimplementation checklist and references

Terminology In this handout, document means one PDF; section usually means one extracted PDF page; chunk means a token-bounded segment written to the vector store; record means the vector-store representation of one chunk.

1. Application purpose and outcomes

The POC demonstrates the smallest complete retrieval pipeline needed for semantic search over local documents. It is intentionally a console application and intentionally uses an in-memory vector store. This keeps the focus on ingestion abstractions, embeddings, and retrieval rather than persistence or web UI concerns.

What the application does

18. Loads provider settings from a local .env file.
19. Discovers every top-level PDF in Data/.
20. Extracts selectable text from each PDF with PdfPig; no OCR is performed.
21. Converts each PDF into an IngestionDocument with page-based sections.
22. Splits documents with Microsoft.Extensions.DataIngestion.DocumentTokenChunker.
23. Generates embeddings through Microsoft.Extensions.AI.
24. Writes chunk content and vectors to Microsoft Semantic Kernel's InMemoryVectorStore connector.
25. Accepts an interactive or one-shot question and returns the top three matches.

What the application deliberately does not do

- No OCR, so image-only or scanned PDFs are skipped with a clear error.
- No persistent database; all vectors disappear when the process exits.
- No answer generation. The output is retrieval evidence: score, document name, and chunk content.
- No web service, authentication, authorization, document upload, or multi-user isolation.

2. Core concepts

Concept	Meaning in this POC
Ingestion document	A format-neutral Microsoft object containing sections and elements. It separates PDF parsing from downstream chunking.
Chunk	A bounded piece of text suitable for embedding and retrieval. Chunks overlap so context near a boundary is not lost.
Embedding	A numeric vector representing semantic meaning. Similar text tends to occupy nearby positions in vector space.
Vector store	A collection that stores content plus vectors and supports nearest-neighbor search.

Concept	Meaning in this POC
Cosine similarity	The configured comparison method. Higher result scores indicate closer semantic alignment.
Provider abstraction	The application consumes IEmbeddingGenerator, while startup chooses Azure OpenAI or Ollama.

Key design idea The reader knows about PDFs, the chunker knows about ingestion documents, and the vector writer knows about chunks. Each stage depends on an abstraction rather than on the internal details of the previous technology.

3. Architecture and end-to-end flow

The application is a linear pipeline with two uses of the embedding generator: first during ingestion to vectorize chunks, then during search to vectorize the user's query.

Stage	Input	Component	Output
1. Configure	.env	EnvFile + AiOptionsValidator	Validated AiOptions
2. Select provider	AiOptions	EmbeddingGeneratorFactory	IEmbeddingGenerator<string, Embedding<float>>
3. Discover	Data/	DocumentSearchService	Ordered FileInfo[]
4. Read	PDF stream	PdfPigDocumentReader	IngestionDocument
5. Chunk	IngestionDocument	DocumentTokenChunker	IAsyncEnumerable<IngestionChunk<string>>
6. Embed + write	Chunks	VectorStoreWriter	In-memory vector records
7. Search	Question text	VectorStoreCollection.SearchAsync	Top 3 scored records

Control flow

Figure 1. Application lifecycle

```

Program.cs
-> load .env
-> build DI container
-> resolve DocumentSearchService
-> IngestAsync(Data/)
    -> probe embedding dimension
    -> read PDF -> normalize -> chunk -> embed -> store
-> SearchAsync(question)
    -> embed question -> vector search -> print top 3
-> dispose pipeline and vector store

```

Why dependency injection matters

Program.cs composes the application but does not implement PDF extraction, provider selection, ingestion, or search. This makes each responsibility testable and replaceable. For example, a future native Microsoft PDF reader could replace PdfPigDocumentReader without changing the chunker or search service.

4. Project structure

Area	File	Responsibility
Entry point	Program.cs	Host, logging, DI, application mode, lifecycle.
Configuration	Configuration/EnvFile.cs	Minimal .env parsing with helpful format/missing-file errors.
Configuration	Configuration/AiOptions.cs	Strongly typed provider settings.
Configuration	Configuration/AiOptionsValidator.cs	Provider-specific required fields and endpoint validation.
Services	Services/EmbeddingGeneratorFactory.cs	Azure or Ollama embedding client creation.
Ingestion	Ingestion/PdfPigDocumentReader.cs	PDF extraction and normalization.
Ingestion	Ingestion/CountingVectorStoreWriter.cs	VectorStoreWriter adapter that counts chunks.
Services	Services/DocumentSearchService.cs	Document discovery, pipeline composition, ingestion, and search.
Project	ProcessDataAI.csproj	Target framework and NuGet dependencies.

5. Dependencies and why they exist

Package	Version	Role
Microsoft.Extensions.DataIngestion	10.7.0-preview.1.26309.5	Document model, chunker, pipeline, result, and vector writer.
Microsoft.Extensions.AI.OpenAI	10.7.0	Adapts OpenAI SDK embedding clients to IEmbeddingGenerator.
Microsoft.Extensions.VectorData	Transitive	Common vector-store collection and search abstractions.
Microsoft.SemanticKernel.Connectors.InMemory	1.74.0-preview	Concrete in-memory VectorStore implementation.
Azure.AI.OpenAI	2.1.0	AzureOpenAIClient for Azure deployments.
PdfPig	0.1.12	Local PDF text extraction without OCR.
Microsoft.ML.Tokenizers.Data.O200kBase	2.0.0	Tokenizer data used by the token chunker.
Microsoft.Extensions.Hosting	10.0.9	Host, DI, application lifetime, and configuration composition.
Microsoft.Extensions.Logging.Console	10.0.9	Structured console logging.
Microsoft.Bcl.Memory	10.0.9	Explicit patched transitive version to avoid a vulnerability warning.

Package references: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\ProcessDataAI.csproj>

Preview packages Microsoft.Extensions.DataIngestion and the in-memory connector are preview dependencies. A production implementation should pin versions, test upgrades deliberately, and expect API changes.

6. Startup and dependency injection

Program.cs owns composition. It loads .env before building the host, adds the values as an in-memory configuration source, configures single-line timestamped logging, registers validated options and singleton services, starts the host, ingests Data/, and then chooses one-shot or interactive search.

Core DI registration

```
HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);
builder.Configuration.AddInMemoryCollection(envValues);

builder.Services.AddSingleton<PdfPigDocumentReader>();
builder.Services.AddSingleton<EmbeddingGeneratorFactory>();
builder.Services.AddSingleton<IEmbeddingGenerator<string, Embedding<float>>>>(
    services => services.GetRequiredService<EmbeddingGeneratorFactory>().Create());
builder.Services.AddSingleton<DocumentSearchService>();
```

Singleton lifetime is appropriate for this POC because one process owns one provider client, one in-memory store, and one ingestion/search session. The host disposes registered disposable services during shutdown.

Startup implementation: <C:\Users\cris\source\repos\MS-ProcessYourData\ProcessDataAI\Program.cs>

7. Environment configuration and validation

Provider selection is data-driven. Switching between Azure and Ollama requires only an .env change; the downstream pipeline always receives the same IEmbeddingGenerator abstraction.

.env.example

```
AI_PROVIDER=Azure

AZURE_OPENAI_ENDPOINT=
AZURE_OPENAI_API_KEY=
AZURE_OPENAI_EMBEDDING_MODEL=

OLLAMA_ENDPOINT=http://192.168.178.22:11434/v1
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
```

Configuration lifecycle

26. EnvFile.Load checks that .env exists and parses non-empty, non-comment KEY=VALUE lines.
27. Program.cs maps configuration keys into AiOptions and nested Azure/Ollama option objects.
28. AiOptionsValidator checks AI_PROVIDER and validates only the selected provider's required fields.
29. ValidateOnStart causes invalid configuration to fail before ingestion begins.

Secret handling The real .env is ignored by Git. .env.example documents keys but must not contain credentials. For a deployed service, prefer a managed secret store or environment injection rather than a local file.

Example configuration: [C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\.env.example](#)

Parser: [C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Configuration\EnvFile.cs](#)

Typed settings: [C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Configuration\AiOptions.cs](#)

Validation: [C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Configuration\AiOptionsValidator.cs](#)

8. Embedding provider abstraction

EmbeddingGeneratorFactory is the only class that knows how to create provider-specific clients. Both branches return IEmbeddingGenerator<string, Embedding<float>>, which is the Microsoft.Extensions.AI contract consumed everywhere else.

Provider selection, shortened

```
if (settings.Provider.Equals(AiOptions.AzureProvider,
    StringComparison.OrdinalIgnoreCase))
{
    var client = new AzureOpenAIClient(
        new Uri(azure.Endpoint),
        new AzureKeyCredential(azure.ApiKey));
    return client.GetEmbeddingClient(azure.EmbeddingModel)
        .AsIEmbeddingGenerator();
}

var options = new OpenAIClientOptions { Endpoint = new Uri(ollama.Endpoint) };
var client = new OpenAIClient(new ApiKeyCredential("not-required"), options);
return client.GetEmbeddingClient(ollama.EmbeddingModel)
    .AsIEmbeddingGenerator();
```

Ollama exposes an OpenAI-compatible /v1 endpoint. The OpenAI SDK requires a credential object, so the factory supplies a placeholder string; Ollama ignores it. No real Ollama API key is read or required.

Provider factory: <C:\Users\cris\source\repos\MS-ProcessYourData\ProcessDataAI\Services\EmbeddingGeneratorFactory.cs>

9. PDF extraction and document normalization

Microsoft.Extensions.DataIngestion defines `IngestionDocumentReader` but does not parse these local PDFs by itself. `PdfPigDocumentReader` supplies the format-specific adapter. It opens the incoming stream, extracts each page in content order, and translates non-empty page text into Microsoft's neutral ingestion document model.

Core reader logic

```
using PdfDocument pdf = PdfDocument.Open(source);
var document = new IngestionDocument(identifier);

foreach (var page in pdf.GetPages())
{
    string text = ContentOrderTextExtractor.GetText(page).Trim();
    if (string.IsNullOrEmpty(text)) continue;

    var section = new IngestionDocumentSection();
    section.Elements.Add(
        new IngestionDocumentParagraph(text) { PageNumber = page.Number });
    document.Sections.Add(section);
}
```

Why ContentOrderTextExtractor is used

PDFs store drawing instructions, not necessarily reading-order paragraphs. `PdfPig`'s content-order extractor is a better default than directly reading `page.Text` because it attempts to reconstruct human reading order.

Image-only PDFs

If every page produces empty text, the reader throws `InvalidDataException`. The pipeline captures that failure as an `IngestionResult`, logs it, skips the document, and continues with other PDFs. This is why the two Jam-export PDFs produce error logs but do not stop a successful `EmployeeHandbook` search.

No OCR by design A PDF can display words while containing only an image. Such a document has no selectable text for `PdfPig` to extract. OCR is a separate capability and is intentionally outside this POC.

PDF reader: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Ingestion\PdfPigDocumentReader.cs>

10. Chunking strategy

The POC uses `DocumentTokenChunker` with the `o200k` tokenizer. `HeaderChunker` was not selected because raw PDF extraction does not reliably identify semantic heading levels. Inventing headings from layout would add fragile heuristics to a minimal sample.

Chunker configuration

```
var chunkerOptions = new IngestionChunkerOptions(  
    TiktokenTokenizer.CreateForModel("gpt-4o"))  
{  
    MaxTokensPerChunk = 500,  
    OverlapTokens = 75  
};  
  
IngestionChunker<string> chunker =  
    new DocumentTokenChunker(chunkerOptions);
```

Setting	Value	Reason
Maximum	500 tokens	Small enough for focused retrieval; large enough for useful paragraph context.
Overlap	75 tokens	Repeats boundary context so an answer near a split is less likely to be lost.
Tokenizer	o200k / gpt-4o mapping	Provides deterministic token counting independent of the embedding provider.

11. Vector storage and chunk writing

The Microsoft VectorStoreWriter requires the embedding dimension when it creates its collection schema. Dimensions differ by model, so the service generates a small probe embedding and reads probe.Length rather than hardcoding a model-specific number.

Dimension probe and store initialization

```
ReadOnlyMemory<float> probe = await embeddingGenerator.GenerateVectorAsync(
    "embedding dimension probe",
    cancellationToken: cancellationToken);

_vectorStore = new InMemoryVectorStore(new()
{
    EmbeddingGenerator = embeddingGenerator
});

_writer = new CountingVectorStoreWriter(
    _vectorStore,
    probe.Length,
    new VectorStoreWriterOptions
    {
        CollectionName = "documents",
        DistanceFunction = DistanceFunction.CosineSimilarity
    });
```

Why the store receives the embedding generator

The vector-data layer can automatically generate vectors from text when writing and when searching with a text query. That lets the application call SearchAsync(query, top: 3) instead of manually generating a query vector.

CountingVectorStoreWriter

VectorStoreWriter does not expose a simple total-chunk counter. The adapter derives from IngestionChunkWriter<string>, streams every chunk to the real writer, increments ChunkCount, and exposes the underlying collection for search. It does not buffer all chunks in memory before forwarding them.

Counting adapter: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Ingestion\CountingVectorStoreWriter.cs>

12. Ingestion orchestration

DocumentSearchService is the application service coordinating filesystem checks, provider verification, vector-store construction, chunker construction, pipeline execution, result handling, metrics, and lifecycle.

Pipeline composition and per-document handling

```
_pipeline = new IngestionPipeline<string>(
    reader,
    chunker,
    _writer,
    loggerFactory: loggerFactory);

await foreach (IngestionResult result in
    _pipeline.ProcessAsync(pdfes, cancellationToken))
{
    if (!result.Succeeded)
    {
        logger.LogError(result.Exception,
            "Failed to ingest PDF {DocumentName}", result.DocumentId);
        continue;
    }
    successfulDocuments++;
}
```

A failed PDF does not abort the complete batch. The service only fails the application when zero PDFs were successfully ingested. This is a deliberate partial-success policy for a directory ingestion job.

Pipeline lifetime The pipeline is stored as a service field rather than a local using variable. Disposing it immediately after `IngestAsync` would dispose the writer before `SearchAsync` runs. `DocumentSearchService` disposes the pipeline and vector store at application shutdown.

13. Semantic search

After at least one document succeeds, SearchAsync uses the writer's dynamic vector-store collection. The collection embeds the query, compares it with stored chunk embeddings using cosine similarity, and yields up to three VectorSearchResult records.

Top-three search and output

```
await foreach (var result in
    _writer.VectorStoreCollection.SearchAsync(
        query, top: 3, cancellationToken: cancellationToken))
{
    string documentId = GetString(result.Record,
        "documentid", "document_id", "documentId", "DocumentId")
        ?? "unknown";
    string content = GetString(result.Record, "content", "Content")
        ?? string.Empty;

    Console.WriteLine($"Score: {result.Score:F4}");
    Console.WriteLine($"Document: {Path.GetFileName(documentId)}");
    Console.WriteLine($"Content: {content}");
}
```

Interpreting the score

The score ranks results within the same search. It should not be treated as a universal correctness probability. Thresholds, if introduced later, should be calibrated on representative questions and documents.

Why the dynamic record uses multiple key spellings

VectorStoreWriter exposes a dynamic Dictionary<string, object?> schema. The current preview produces lower-case documentid and content keys. GetString accepts several variants to tolerate minor naming changes across preview versions.

Ingestion and search service: [C:\Users\cris\source\repos\MS-ProcessYourData\ProcessDataAI\Services\DocumentSearchService.cs](#)

14. Running and validating the application

Prerequisites

- .NET 10 SDK.
- At least one text-bearing PDF in Data/.
- An Ollama embedding model available at the configured endpoint, or a deployed Azure OpenAI embedding model.
- A .env file copied from .env.example and completed for the selected provider.

Build and run

```
dotnet restore
dotnet build
dotnet run

# One-shot query
dotnet run -- --query "How many vacation days per calendar year?"
```

Expected one-shot output

Example result; score can vary by model

```
Score: 0.7703
Document: EmployeeHandbook.pdf
Content: Employee Handbook
Vacation Policy
Full-time employees receive 25 paid vacation days per calendar year.
Unused vacation days may be carried over up to a maximum of five days.
```

Without `--query`, the application repeatedly prompts Question (or 'exit'): and reuses the same in-memory collection until the user exits.

15. Errors, logging, and troubleshooting

Symptom	Cause	Action
.env not found	Startup configuration file is missing.	Copy .env.example to .env and configure one provider.
AI_PROVIDER must be Azure or Ollama	Unsupported or misspelled provider value.	Use exactly Azure or Ollama; comparison is case-insensitive.
Missing required provider configuration	A selected-provider field is empty.	Complete endpoint, model/deployment, and Azure key when applicable.
Data directory was not found	Application started from the wrong working directory or Data/ is missing.	Run from ProcessDataAI or create Data/.
Data contains no PDF files	No top-level *.pdf files were discovered.	Add text-bearing PDFs; subdirectories are not searched.
Contains no extractable text	The PDF is image-only, scanned, or blank.	Use a text PDF, remove it, or add an OCR stage in a future version.
Embedding generation failed	Endpoint, model, credentials, network, or Ollama availability problem.	Check .env, confirm the model exists, and test endpoint reachability.
No PDFs could be ingested	Every discovered PDF failed reading or writing.	Review the preceding per-document errors.
Semantic search failed	Query embedding or vector search failed.	Check provider health and ensure ingestion completed.

Why PDF errors appear twice

The Microsoft IngestionPipeline logs the exception when reading fails. DocumentSearchService then logs the unsuccessful IngestionResult. Both refer to the same failure. They are fail-level log entries, but they are non-fatal when another document succeeds.

Useful success logs

- Loaded AI provider confirms provider selection.
- Discovered N PDF file(s) confirms directory scanning.
- Extracted text from N page(s) confirms PdfPig normalization.
- Ingestion completed reports successful documents, chunk count, and elapsed time.

- Executing semantic search confirms query execution.

16. Limitations and production evolution

POC limitation	Production direction
In-memory vectors	Use a supported persistent vector database and define migration/backup behavior.
Reingests on every run	Add document fingerprints, incremental ingestion, and deletion/update handling.
No OCR	Insert an OCR/document-intelligence reader for scanned documents, subject to cost and privacy requirements.
One process, one user	Expose a service API with authentication, tenant isolation, quotas, and cancellation.
No generated answer	Optionally add a chat model that answers only from retrieved chunks and includes citations.
No retrieval evaluation	Build a test set of questions and expected documents; measure recall, ranking, and latency.
Dynamic record schema	Use an explicit typed vector record when the preview API and target provider allow it.
Preview packages	Pin versions, monitor release notes, add integration tests, and budget for API migrations.
Local .env secrets	Use managed identity, workload identity, Key Vault, or deployment-managed secrets.

17. Reimplementation checklist and references

Rebuild the POC from an existing console project

30. Add the required NuGet packages and remove any persistent database connector not needed by the POC.
31. Create .env.example, ignore .env, load it before host construction, and validate typed options at startup.
32. Return one IEmbeddingGenerator implementation from a provider factory selected by AI_PROVIDER.
33. Implement IngestionDocumentReader for the source format; create one IngestionDocument per PDF and page-based sections.
34. Choose a Microsoft Data Ingestion chunker. Use token chunking when source headings are not trustworthy.
35. Generate a probe embedding to discover dimensions, then construct InMemoryVectorStore and VectorStoreWriter.
36. Compose IngestionPipeline and process all PDFs while allowing partial success.
37. Keep pipeline/writer/store alive through the search phase and dispose them at shutdown.
38. Call SearchAsync with text and top: 3; print score, source name, and content.
39. Verify restore, warning-free build, PDF discovery, extraction, chunk count, embeddings, and a known-answer search.

Project file links

Entry point: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Program.cs>

Project dependencies: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\ProcessDataAI.csproj>

Environment example: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\.env.example>

Typed configuration: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Configuration\AiOptions.cs>

Configuration validator: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Configuration\AiOptionsValidator.cs>

Environment parser: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Configuration\EnvFile.cs>

Provider factory: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Services\EmbeddingGeneratorFactory.cs>

PDF reader: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Ingestion\PdfPigDocumentReader.cs>

Counting writer: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Ingestion\CountingVectorStoreWriter.cs>

Ingestion/search service: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\Services\DocumentSearchService.cs>

Quick README: <C:\Users\crist\source\repos\MS-ProcessYourData\ProcessDataAI\README.md>

Official and package references

[Microsoft Data Ingestion quickstart](#)

[Microsoft Vector Stores guide](#)

[Microsoft.Extensions.DataIngestion API](#)

[Microsoft.Extensions.DataIngestion NuGet](#)

[In-memory vector connector NuGet](#)

[PdfPig NuGet and usage](#)

Final mental model

Remember the boundary PdfPig turns a PDF into Microsoft's neutral document model. Microsoft Data Ingestion turns that document into chunks. Microsoft.Extensions.AI turns text into vectors. Microsoft.Extensions.VectorData and InMemoryVectorStore store and retrieve those vectors. DocumentSearchService wires the stages together.